

HDI PREDICTION SYSTEM

A COMPREHENSIVE MEASURE OF WELL-BEING



A Comprehensive Measure of Well-Being

Project Design Phase-II: Technology Stack (Architecture & Stack) – HDI Prediction System

Authors	Paruchuri Venkatesh; Rokkam Guna Sekhar
Document Type	Technical Design (Phase-II)
Date	2026

1. Introduction to Phase-II

This Phase-II document defines the **Technology Stack** and **System Architecture** for the HDI Prediction System, a Machine Learning-powered web application that predicts a country's Human Development Index (HDI) from key socio-economic indicators. It formalizes the architectural layers, the responsibilities of each layer, and the specific tools selected to implement the platform as a secure, maintainable, and deployable academic project system.

Phase-II is focused on ensuring the solution is technically coherent end-to-end: from the user interface and authentication, through business rules and model inference, to persistent storage and analytics-ready history tracking.

2. Purpose of the Technology Stack

The technology stack specifies the set of frameworks, libraries, and runtime components used to build the HDI Prediction System. Its purpose is to ensure: (i) consistent development practices, (ii) predictable deployment and maintenance, and (iii) reliable system behavior across data ingestion, model execution, and result persistence.

In this project, the stack is selected to balance **academic clarity**, **implementation simplicity**, and **real-world relevance** while supporting core platform requirements such as authentication, input validation, low-latency prediction, and traceable prediction history.

Key Design Goal – Separate concerns across layers so that UI changes, business-rule updates, and model improvements can be developed and tested with minimal cross-impact.

3. Architecture Overview

The system follows a layered web-application architecture with a dedicated Machine Learning layer for preprocessing and inference. Users interact via a browser-based interface; requests are handled by a Flask application; business rules validate inputs and coordinate prediction; the trained model generates an HDI score; and results are stored in a SQLite database through an ORM for later review and analytics.

Component	Primary Responsibility
Web UI	User input, result presentation, dashboard visual summaries
Flask Application	Routing, sessions, authentication integration, request/response lifecycle
Business Services	Validation rules, prediction orchestration, persistence of history
ML Inference	Feature preparation, model loading, HDI prediction output
Data Layer	ORM models, queries, transaction handling, SQLite storage

4. Layer-by-Layer System Architecture

This section defines each layer, its scope, and how it interacts with other layers to deliver HDI predictions and analytics-ready history.

4.1 Presentation Layer

The Presentation Layer is responsible for the user experience and includes web pages, forms, and responsive layouts. It collects indicator inputs (e.g., life expectancy and schooling metrics) and displays predicted HDI results, category labels, and dashboard summaries.

4.2 Application Layer

The Application Layer is implemented using Flask and manages HTTP routing, controller actions, template rendering, session handling, and integration with authentication mechanisms. It acts as the gateway between the UI and business services.

4.3 Business Logic Layer

The Business Logic Layer enforces system rules such as input constraints, permissible ranges, and the steps for producing and saving a prediction. It coordinates calls to the ML layer and the data access layer, ensuring outcomes are consistent and auditable (e.g., by storing prediction history).

4.4 Machine Learning Layer

The Machine Learning Layer performs preprocessing and inference. It loads the serialized model, applies the expected feature ordering and transformations, runs prediction, and returns a normalized HDI score (0.000–1.000) and its development category (Low/Medium/High/Very High) according to the project's classification thresholds.

4.5 Data Access Layer

The Data Access Layer abstracts database operations through SQLAlchemy ORM. It defines models (users, predictions, audit-like records), encapsulates queries, and reduces risk from manual SQL handling while improving maintainability and testability.

4.6 Database Layer

The Database Layer persists application and prediction data in SQLite. It stores user accounts (with hashed credentials), prediction inputs, predicted HDI outputs, timestamps, and derived categories to support dashboards and history review.

4.7 Utilities

Utilities include configuration management, logging, error handling, and model serialization support. These functions provide operational consistency across environments and improve debugging, traceability, and controlled evolution of the model and application settings.

5. Technology Stack (Concise)

The following stack is selected to implement the HDI Prediction System with clear separation of concerns, rapid development, and reproducible results.

Area	Technologies	Role in System
Frontend	HTML5, CSS3, Bootstrap, Jinja2	Responsive pages, forms, layout system, server-side templating
Backend	Python, Flask, Flask-Login	Web server logic, routing, sessions, user authentication flows
Machine Learning	Scikit-Learn, Pandas, NumPy	Data handling, feature preparation, model training/inference pipeline
Database	SQLite, SQLAlchemy	Persistent user and prediction storage; ORM-based access and queries
Model Storage	Pickle	Serialize and load trained model artifacts for runtime inference
Visualization	Matplotlib	Charts/plots for dashboards and distribution summaries
Testing & Environment	PyTest, Virtual Environment	Unit/integration tests; dependency isolation and reproducibility

Assumption — The runtime prediction path must use the same feature schema as training (ordering, scaling decisions, and categorical handling) to prevent silent accuracy degradation.

6. Application Workflow

The platform operates through a defined user journey to ensure predictions are authenticated, stored, and reflected in analytics views.

1. Register
2. Login
3. Dashboard
4. Predict
5. ML Model
6. Result
7. Save History
8. Dashboard Update
9. View History

7. Security Considerations

Security is addressed as a baseline requirement because the system stores user accounts and prediction history. The following measures are incorporated at design level to reduce common web-application risks.

- **Authentication & sessions** — Flask-Login manages user sessions; credentials are stored as salted, hashed values (e.g., Werkzeug hashing) rather than plain text.
- **Input validation** — Client-side and server-side checks constrain indicator ranges and reject incomplete or malformed submissions to protect model integrity and system stability.
- **Database safety** — SQLAlchemy ORM promotes parameterized interactions and reduces injection risks compared with manual query construction.
- **Least exposure of artifacts** — Model files and configuration secrets are not exposed through the UI; access is controlled via server-side file permissions and application configuration.
- **Operational logging** — Structured logs support debugging and traceability of failures in authentication, prediction, and persistence steps while avoiding sensitive data leakage.

8. Conclusion: Architectural Benefits

This Phase-II architecture provides a structured foundation for an HDI prediction platform that is maintainable, testable, and suitable for academic evaluation. Layer separation reduces coupling between the web interface, core prediction logic, and database operations, enabling iterative improvement (e.g., model upgrades or new dashboard metrics) without redesigning the entire system.

By combining Flask-based application structure, a dedicated ML inference layer, ORM-backed persistence, and defined security practices, the platform supports reliable real-time HDI predictions, consistent history tracking, and analytics-friendly reporting in a clean and extensible technical design.